

Informe – Entorno UEFI, Desarrollo y Análisis de Seguridad

Materia: Sistemas de Computación **Alumnos:**

- Martina Juri
- Marcos Morán
- Francisco Gomez Neimann
- Cristian Eduardo Arteaga Barrera

Vínculo al repositorio: <https://github.com/MMoran2001/Electrotonto-y-Computarados>

Trabajo Práctico: TP3.a - UEFI

1. Introducción

El presente trabajo tiene como objetivo comprender la arquitectura de la Interfaz de Firmware Extensible Unificada (UEFI), analizando su funcionamiento como entorno previo al sistema operativo (pre-OS).

Se desarrollaron aplicaciones nativas en este entorno, se exploró la interacción con el hardware mediante protocolos, y se realizaron pruebas tanto en entornos virtualizados como en hardware físico.

Asimismo, se abordaron aspectos de seguridad, incluyendo el análisis de memoria, variables persistentes y ejecución de código a bajo nivel.

2. Preparación del Entorno

Se configuró un entorno de trabajo en Linux, instalando herramientas necesarias como:

- QEMU (emulación de hardware)
- OVMF (firmware UEFI open-source)
- gnu-efi (toolchain para desarrollo UEFI)
- Ghidra (ingeniería inversa)

Comandos utilizados

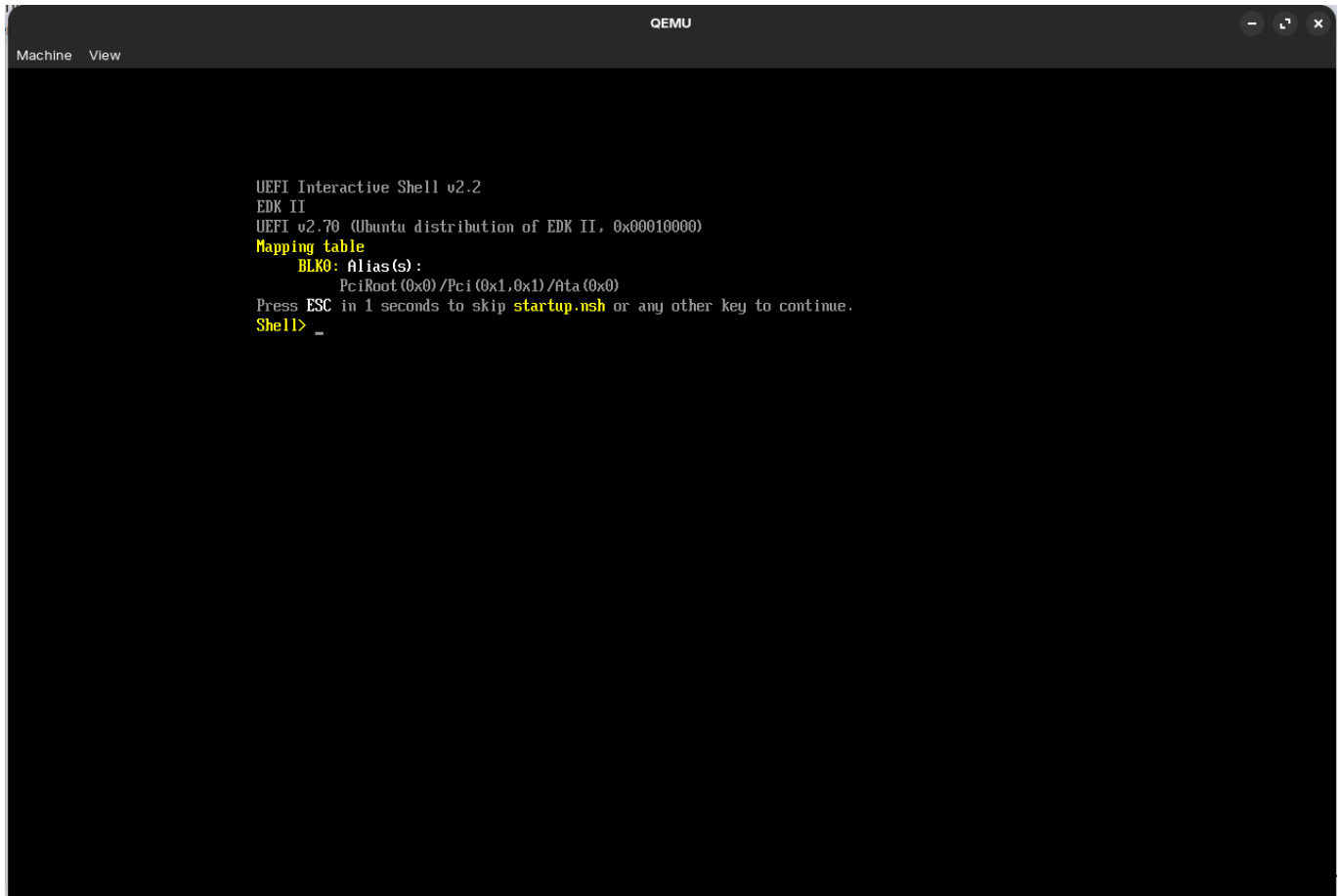
```
mkdir -p ~/uefi_security_lab && cd ~/uefi_security_lab
sudo apt update
sudo apt install -y qemu-system-x86 ovmf gnu-efi build-essential binutils-mingw-
w64
sudo apt install -y ghidra
```

3. Trabajo Práctico 1: Exploración del entorno UEFI

3.1 Arranque en entorno virtual

Se ejecutó QEMU con firmware UEFI:

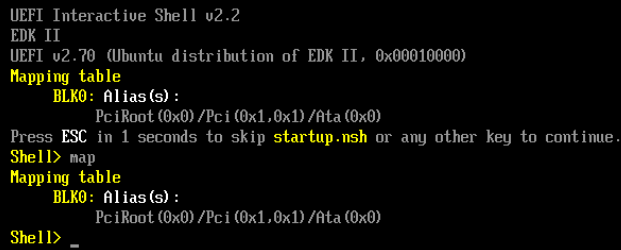
```
qemu-system-x86_64 -m 512 -bios /usr/share/ovmf/OVMF.fd -net none
```



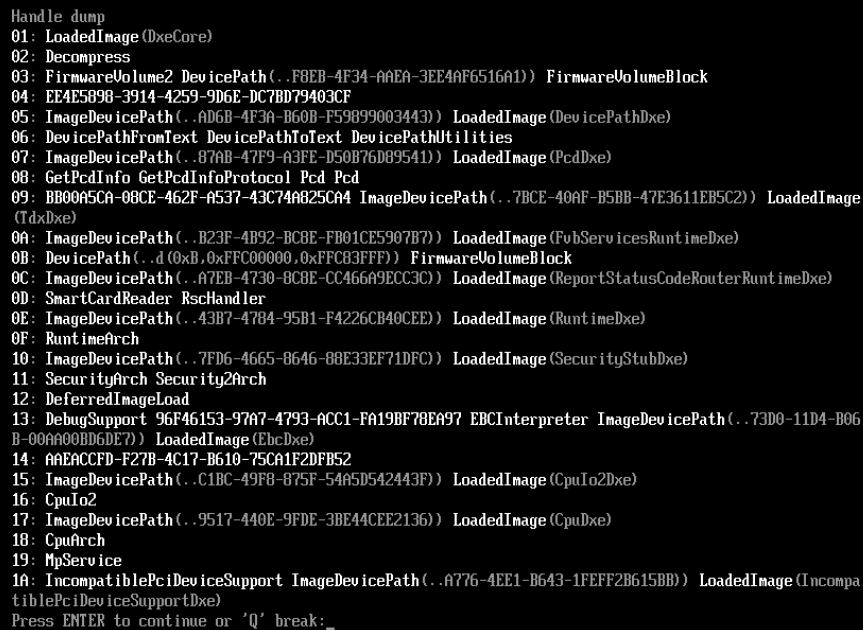
3.2 Exploración de dispositivos

Comandos utilizados:

```
map
FS0:
ls
dh -b
```



Captura de pantalla del comando map



Captura de pantalla del comando dh -b

¿Cuál es la ventaja de seguridad y compatibilidad frente al BIOS?

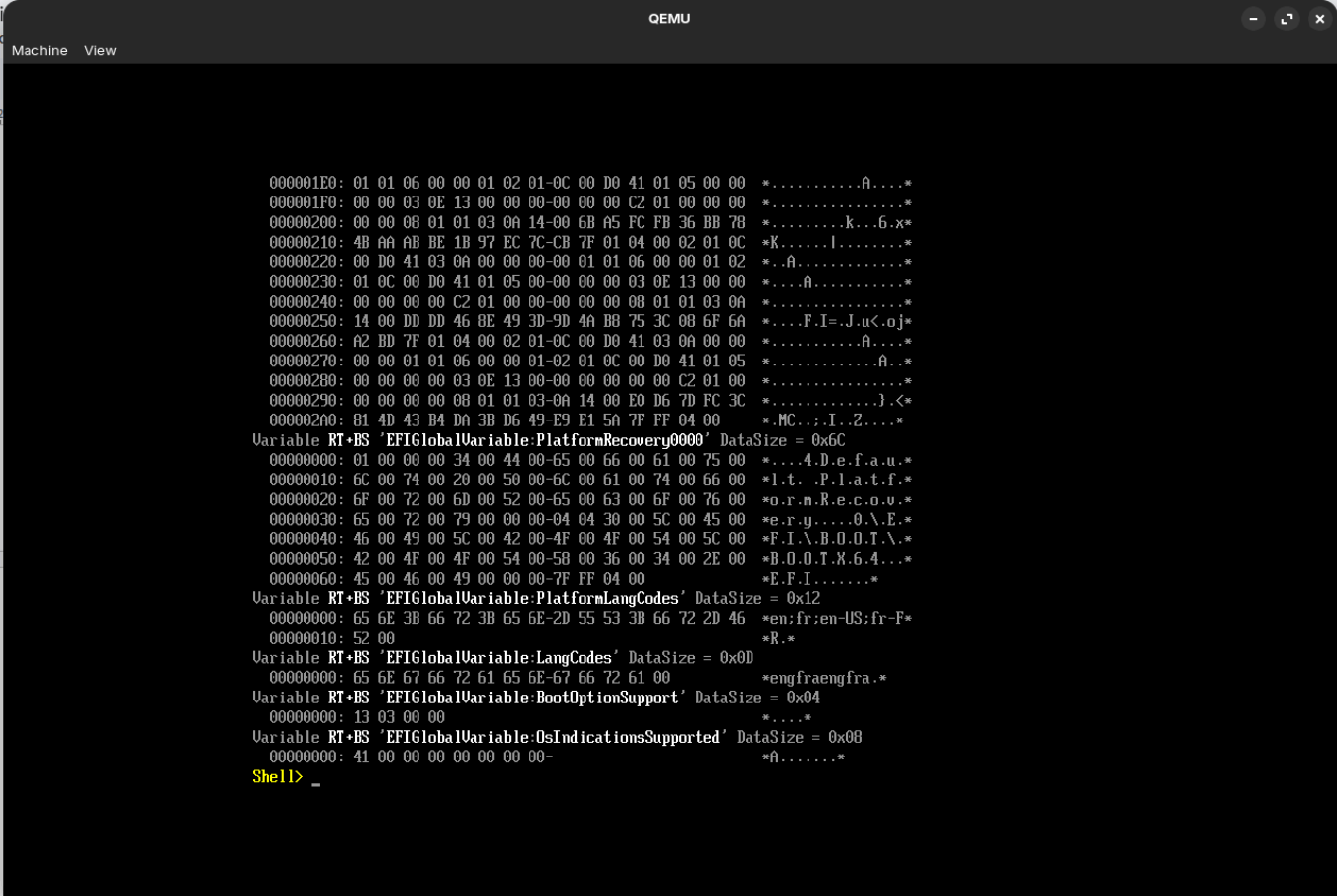
UEFI utiliza un modelo basado en **handles y protocolos**, lo que permite abstraer el hardware y evitar dependencias directas de direcciones físicas o puertos específicos.

Esto mejora la compatibilidad entre distintos dispositivos y plataformas, y aumenta la seguridad al evitar accesos directos no controlados al hardware, reduciendo la superficie de ataque.

3.3 Variables globales (NVRAM)

Comandos:

```
dmpstore
set TestSeguridad "Hola UEFI"
set -v
```

A screenshot of a QEMU terminal window. The window title is 'QEMU'. Inside the terminal, a memory dump is displayed, showing hexadecimal addresses and their corresponding ASCII values. The dump includes several EFI Global Variables: 'PlatformRecovery' (DataSize = 0x6C), 'PlatformLangCodes' (DataSize = 0x12), 'LangCodes' (DataSize = 0x0D), 'BootOptionSupport' (DataSize = 0x04), and 'OsIndicationsSupported' (DataSize = 0x0B). The terminal also shows the 'Shell' prompt and a cursor.

Captura de pantalla del comando dmpstore

```

Shell> dmpstore Boot*
Variable NU*RT*BS 'EFIGlobalVariable:BootOrder' DataSize = 0x04
00000000: 00 00 01 00 *....*
Variable NU*RT*BS 'EFIGlobalVariable:Boot0001' DataSize = 0x58
00000000: 01 00 00 00 2C 00 45 00 46 00 49 00 20 00 49 00 *....E.F.I. .I.*
00000010: 6E 00 74 00 65 00 72 00 6E 00 61 00 6C 00 20 00 *n.t.e.r.n.a.l. .*
00000020: 53 00 68 00 65 00 6C 00 6C 00 00 00 04 07 14 00 *S.h.e.l.l.....*
00000030: C9 BD B8 7C EB F8 34 4F AA EA 3E E4 AF 65 16 A1 *...l..40..>.e..*
00000040: 04 06 14 00 83 A5 04 7C 3E 9E 1C 4F AD 65 E0 52 *.....l>..0.e.R*
00000050: 68 D0 B4 D1 7F FF 04 00 *h.....*
Variable NU*RT*BS 'EFIGlobalVariable:Boot0000' DataSize = 0x3E
00000000: 09 01 00 00 2C 00 55 00 69 00 41 00 70 00 70 00 *....U.i.A.p.p.*
00000010: 00 00 04 07 14 00 C9 BD B8 7C EB F8 34 4F AA EA *.....l..40..*
00000020: 3E E4 AF 65 16 A1 04 06 14 00 21 AA 2C 46 14 76 *>..e.....f..F.u*
00000030: 03 45 83 6E 8A B6 F4 66 23 31 7F FF 04 00 *E.n...f#1....*
Variable RT*BS 'EFIGlobalVariable:BootCurrent' DataSize = 0x02
00000000: 01 00 *..*
Variable RT*BS 'EFIGlobalVariable:BootOptionSupport' DataSize = 0x04
00000000: 13 03 00 00 *....*
Shell> _

```

BootOrder del entorno en QEMU

```

Machine View

00000280: 00 00 00 00 03 0E 13 00 00 00 00 00 00 C2 01 00 *.....*
00000290: 00 00 00 00 08 01 01 03 0A 14 00 E0 D6 7D FC 3C *.....>.<.*
000002A0: 81 4D 43 B4 DA 3B D6 49 E9 E1 5A 7F FF 04 00 *MC...I.Z....*
Variable RT*BS 'EFIGlobalVariable:PlatformRecovery0000' DataSize = 0x6C
00000000: 01 00 00 00 34 00 44 00 65 00 66 00 61 00 75 00 *...4.D.e.f.a.u.*
00000010: 6C 00 74 00 20 00 50 00 6C 00 61 00 74 00 66 00 *l.t. .P.l.a.t.f.*
00000020: 6F 00 72 00 6D 00 52 00 65 00 63 00 6F 00 76 00 *o.r.m.R.e.c.o.v.*
00000030: 65 00 72 00 79 00 00 00 04 04 30 00 5C 00 45 00 *e.r.y.....0.\E.*
00000040: 46 00 49 00 5C 00 42 00 4F 00 4F 00 54 00 5C 00 *P.I.\B.O.O.T.\.*
00000050: 42 00 4F 00 4F 00 54 00 58 00 36 00 34 00 2E 00 *B.O.O.T.X.6.4...*
00000060: 45 00 46 00 49 00 00 00 7F FF 04 00 *E.F.I.....*
Variable RT*BS 'EFIGlobalVariable:PlatformLangCodes' DataSize = 0x12
00000000: 65 6E 3B 66 72 3B 65 6E 2D 55 53 3B 66 72 2D 46 *en;fr;en-US;fr-F*
00000010: 52 00 *R.*
Variable RT*BS 'EFIGlobalVariable:LangCodes' DataSize = 0x0D
00000000: 65 6E 67 66 72 61 65 6E 67 66 72 61 00 *engfraengfra.*
Variable RT*BS 'EFIGlobalVariable:BootOptionSupport' DataSize = 0x04
00000000: 13 03 00 00 *....*
Variable RT*BS 'EFIGlobalVariable:OsIndicationsSupported' DataSize = 0x0B
00000000: 41 00 00 00 00 00 00 00 00 00 00 00 00 *A.....*
Shell> set TestSeguridad "Hola UEFI"
Shell> set -v
nonesting = False
debuglasterror = 0x0
lasterror = 0x0
profiles = :Driver1;Debug1;network1;network2;
uefishellsupport = 3
uefishellversion = 2.2
uefi version = 2.70
TestSeguridad = Hola UEFI
Shell> _

```

BootOrder del entorno en QEMU

¿Cómo determina el Boot Manager la secuencia de arranque?

El Boot Manager utiliza las variables **Boot####** junto con **BootOrder**, donde:

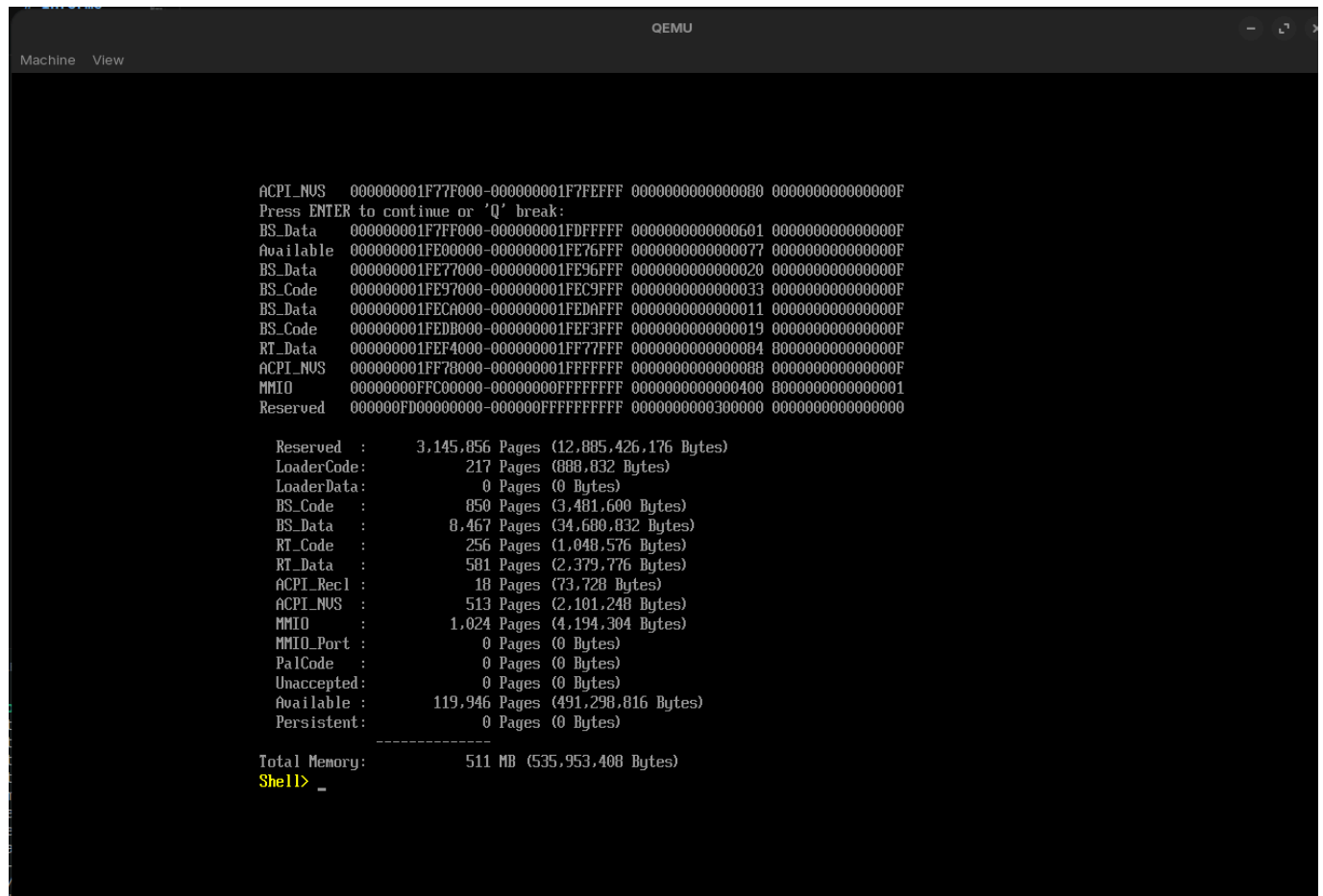
- **Boot####** representa entradas individuales de arranque
- **BootOrder** define el orden en que se intentan

De esta forma, el firmware decide qué dispositivo o aplicación ejecutar primero. En el caso de que una opción falle en arrancar, se pasa a la siguiente.

3.4 Análisis de memoria y hardware

Comandos:

```
memmap -b
pci -b
drivers -b
```



```

Machine  View

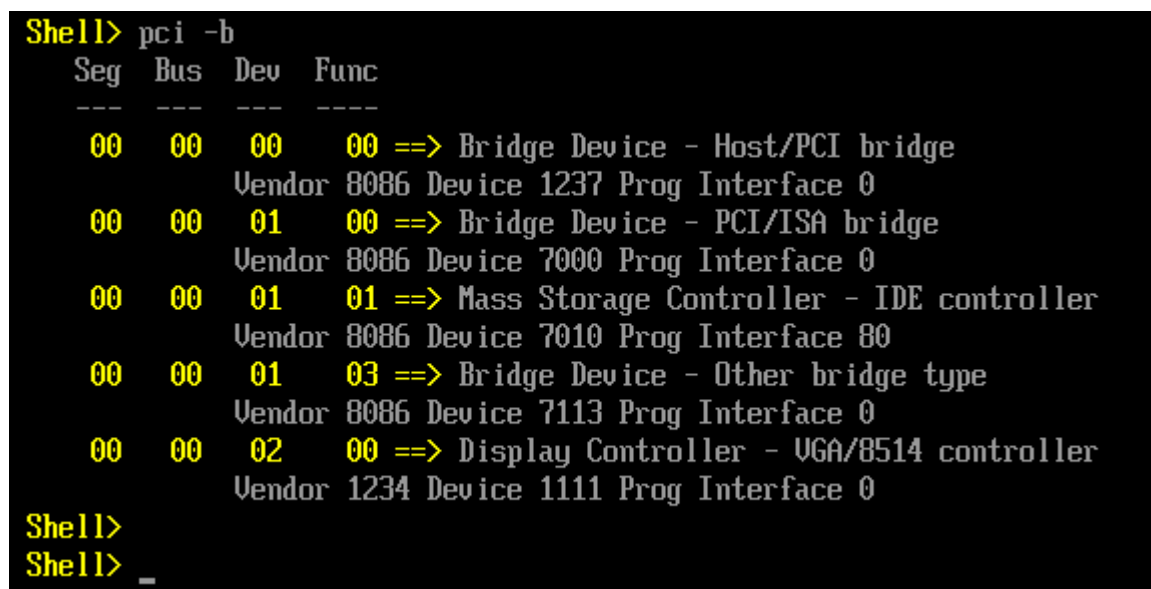
ACPI_NUS  000000001F77F000-000000001F7FEFFF 0000000000000000 000000000000000F
Press ENTER to continue or 'Q' break:
BS_Data  000000001F7FF000-000000001FDF7FFF 0000000000000000 000000000000000F
Available 000000001FE00000-000000001FE76FFF 0000000000000000 000000000000000F
BS_Data  000000001FE77000-000000001FE96FFF 0000000000000000 000000000000000F
BS_Code  000000001FE97000-000000001FEC9FFF 0000000000000000 000000000000000F
BS_Data  000000001FECB000-000000001FEDAFFF 0000000000000000 000000000000000F
BS_Code  000000001FEDB000-000000001FEF3FFF 0000000000000000 000000000000000F
RT_Data  000000001FEF4000-000000001FF77FFF 0000000000000000 800000000000000F
ACPI_NUS 000000001FF78000-000000001FFFFFFF 0000000000000000 000000000000000F
MMIO     00000000FFC00000-00000000FFFFFFFF 0000000000000000 8000000000000001
Reserved 000000FD00000000-000000FFFFFFFF 0000000000000000 0000000000000000

Reserved :      3,145,856 Pages (12,885,426,176 Bytes)
LoaderCode:      217 Pages (888,832 Bytes)
LoaderData:       0 Pages (0 Bytes)
BS_Code :        850 Pages (3,481,600 Bytes)
BS_Data :      8,467 Pages (34,680,832 Bytes)
RT_Code :        256 Pages (1,048,576 Bytes)
RT_Data :        581 Pages (2,379,776 Bytes)
ACPI_Recl :       18 Pages (73,728 Bytes)
ACPI_NUS :       513 Pages (2,101,248 Bytes)
MMIO :          1,024 Pages (4,194,304 Bytes)
MMIO_Port :       0 Pages (0 Bytes)
PalCode :         0 Pages (0 Bytes)
Unaccepted:       0 Pages (0 Bytes)
Available :     119,946 Pages (491,298,816 Bytes)
Persistent:       0 Pages (0 Bytes)

-----
Total Memory:      511 MB (535,953,408 Bytes)
Shell> _

```

Captura del comando mmap -b



```

Shell> pci -b
Seg  Bus  Dev  Func
---  ---  ---  ----
00   00   00   00 ==> Bridge Device - Host/PCI bridge
      Vendor 8086 Device 1237 Prog Interface 0
00   00   01   00 ==> Bridge Device - PCI/ISA bridge
      Vendor 8086 Device 7000 Prog Interface 0
00   00   01   01 ==> Mass Storage Controller - IDE controller
      Vendor 8086 Device 7010 Prog Interface 80
00   00   01   03 ==> Bridge Device - Other bridge type
      Vendor 8086 Device 7113 Prog Interface 0
00   00   02   00 ==> Display Controller - VGA/8514 controller
      Vendor 1234 Device 1111 Prog Interface 0

Shell>
Shell> _

```

Captura del comando pci-b

```

Machine View

QEMU

D      T D
R      Y C I
U      P F A
VERSION E G G #D #C DRIVER NAME          IMAGE NAME
=====
59 0000000A B - - 1 5 PCI Bus Driver        PciBusDxe
5B 00000010 ? - - - - Virtio PCI Driver    VirtioPciDeviceDxe
5C 00000010 ? - - - - Virtio 1.0 PCI Driver Virtio10
5D 00000010 ? - - - - Virtio Block Driver  VirtioBlkDxe
5E 00000010 ? - - - - Virtio SCSI Host Driver  VirtioScsiDxe
5F 00000010 ? - - - - Virtio Random Number Generator Driv VirtioRngDxe
60 00000010 ? - - - - Virtio Serial Driver      VirtioSerialDxe
61 0000000A D - - 2 - Platform Console Management Driver ConPlatformDxe
62 0000000A D - - 2 - Platform Console Management Driver ConPlatformDxe
63 0000000A B - - 2 2 Console Splitter Driver  ConSplitterDxe
64 0000000A ? - - - - Console Splitter Driver  ConSplitterDxe
65 0000000A ? - - - - Console Splitter Driver  ConSplitterDxe
66 0000000A B - - 2 2 Console Splitter Driver  ConSplitterDxe
67 0000000A B - - 1 1 Console Splitter Driver  ConSplitterDxe
6B 0000000A D - - 1 - Graphics Console Driver  GraphicsConsoleDxe
6C 0000000A B - - 1 1 Serial Terminal Driver  TerminalDxe
6D 0000000A D - - 1 - Generic Disk I/O Driver  DiskIoDxe
6E 0000000B ? - - - - Partition Driver (MBR/GPT/El Torito) PartitionDxe
71 0000000A B - - 1 1 SCSI Bus Driver        ScsiBus
72 0000000A D - - 1 - Scsi Disk Driver        ScsiDisk
73 0000000A D - - 1 - Sata Controller Init Driver  SataController
74 00000010 D - - 1 - AtaAtapiPassThru Driver  AtaAtapiPassThruDxe
75 00000010 D - - 1 - ATA Bus Driver        AtaBusDxe
76 00000010 ? - - - - NUM Express Driver    NumExpressDxe
77 00000010 B - - 1 3 OVMF Sio Bus Driver    SioBusDxe
Press ENTER to continue or 'Q' break:

```

```

Machine View

QEMU

83 0000000A ? - - - - DHCP Protocol Driver    Dhcp4Dxe
84 0000000A ? - - - - IP4 Network Service Driver  Ip4Dxe
85 0000000A ? - - - - UDP Network Service Driver  Udp4Dxe
86 0000000A ? - - - - MTFTP4 Network Service      Mtftp4Dxe
87 0000000A ? - - - - DHCP6 Protocol Driver      Dhcp6Dxe
88 0000000A ? - - - - IP6 Network Service Driver  Ip6Dxe
89 0000000A ? - - - - UDP6 Network Service Driver  Udp6Dxe
8A 0000000A ? - - - - MTFTP6 Network Service Driver Mtftp6Dxe
8B 0000000A ? - - - - TCP Network Service Driver  TcpDxe
8C 0000000A ? - - - - TCP Network Service Driver  TcpDxe
8D 0000000A ? - - - - UEFI PXE Base Code Driver    UefiPxeBcdDxe
8E 0000000A ? - - - - UEFI PXE Base Code Driver    UefiPxeBcdDxe
91 00000000 ? - - - - DNS Network Service Driver    DnsDxe
92 00000000 ? - - - - DNS Network Service Driver    DnsDxe
93 0000000A ? - - - - HttpDxe                      HttpDxe
94 0000000A ? - - - - HttpDxe                      HttpDxe
95 0000000A ? - - - - UEFI HTTP Boot Driver        HttpBootDxe
96 0000000A ? - - - - UEFI HTTP Boot Driver        HttpBootDxe
97 0000000A ? - - - - iSCSI Driver                 IScsiDxe
98 0000000A ? - - - - iSCSI Driver                 IScsiDxe
9A 00000010 ? - - - - Virtio Network Driver        VirtioNetDxe
Press ENTER to continue or 'Q' break:
9B 00000020 ? - - - - Usb Uhci Driver              UhciDxe
9C 00000030 ? - - - - Usb Ehci Driver              EhciDxe
9D 00000030 ? - - - - Usb Xhci Driver              XhciDxe
9E 0000000A ? - - - - Usb Bus Driver               UsbBusDxe
9F 0000000A ? - - - - Usb Keyboard Driver          UsbKbdDxe
A0 00000011 ? - - - - Usb Mass Storage Driver      UsbMassStorageDxe
A1 00000010 B - - 1 1 QEMU Video Driver      QemuVideoDxe
A2 00000010 ? - - - - Virtio GPU Driver            VirtioGpuDxe
She11> _

```

Capturas del comando drivers-b

¿Por qué RuntimeServicesCode es un objetivo para malware?

Porque esas regiones no desaparecen completamente cuando termina el entorno de prearranque. Cuando el sistema operativo llama a `ExitBootServices()`, la mayoría de los servicios de arranque dejan de estar disponibles, pero los Runtime Services siguen existiendo para funciones como acceso a variables UEFI, reloj y otros servicios básicos.

Entonces, si un atacante logra modificar código o datos ubicados en regiones de runtime, puede intentar mantener presencia incluso después de que el sistema operativo haya arrancado. Por eso son atractivas para bootkits: están en una zona temprana, privilegiada y persistente del proceso de arranque.

4. Trabajo Práctico 2: Desarrollo de aplicación UEFI

4.1 Código fuente

```
#include <efi.h>
#include <efilib.h>

EFI_STATUS efi_main(EFI_HANDLE ImageHandle, EFI_SYSTEM_TABLE *SystemTable) {
    InitializeLib(ImageHandle, SystemTable);
    SystemTable->ConOut->OutputString(SystemTable->ConOut, L"Iniciando analisis de
seguridad...\r\n");

    unsigned char code[] = { 0xCC };

    if (code[0] == 0xCC) {
        SystemTable->ConOut->OutputString(SystemTable->ConOut, L"Breakpoint
estatico alcanzado.\r\n");
    }

    return EFI_SUCCESS;
}
```

¿Por qué utilizamos `SystemTable->ConOut->OutputString` en lugar de la función `printf` de C?

Porque una aplicación UEFI no corre dentro de Linux ni dentro de Windows. Corre antes del sistema operativo, en un entorno donde no existe la biblioteca estándar de C como normalmente la usamos.

`printf` depende de un runtime, de una salida estándar, de llamadas al sistema operativo, buffers, terminal, etc. En UEFI todavía no tenemos eso.

Esa función pertenece a los servicios de consola que ofrece el firmware UEFI. Es la manera "nativa" de imprimir texto en pantalla dentro del entorno pre-OS.

4.2 Compilación

Compilar a código objeto


```
gcc -I/usr/include/efi -I/usr/include/efi/x86_64 -I/usr/include/efi/protocol \
-fpic -ffreestanding -fno-stack-protector -fno-strict-aliasing \
-fshort-wchar -mno-red-zone -maccumulate-outgoing-args -Wall \
-c -o aplicacion.o aplicacion.c
```

Linkear (generar .so intermedio)

```
ld -shared -Bsymbolic -L/usr/lib -L/usr/lib/efi \
-T /usr/lib/elf_x86_64_efi.lds \
/usr/lib/crt0-efi-x86_64.o aplicacion.o \
-o aplicacion.so -lEFI -lgnuEFI
```

Convertir a ejecutable EFI (PE/COFF)

```
objcopy -j .text -j .sdata -j .data -j .dynamic -j .dynsym \
-j .rel -j .rela -j .rel.* -j .rela.* -j .reloc \
--target=efi-app-x86_64 aplicacion.so aplicacion.efi
```

4.3 Análisis de Metadatos y Decompilación

Comandos:

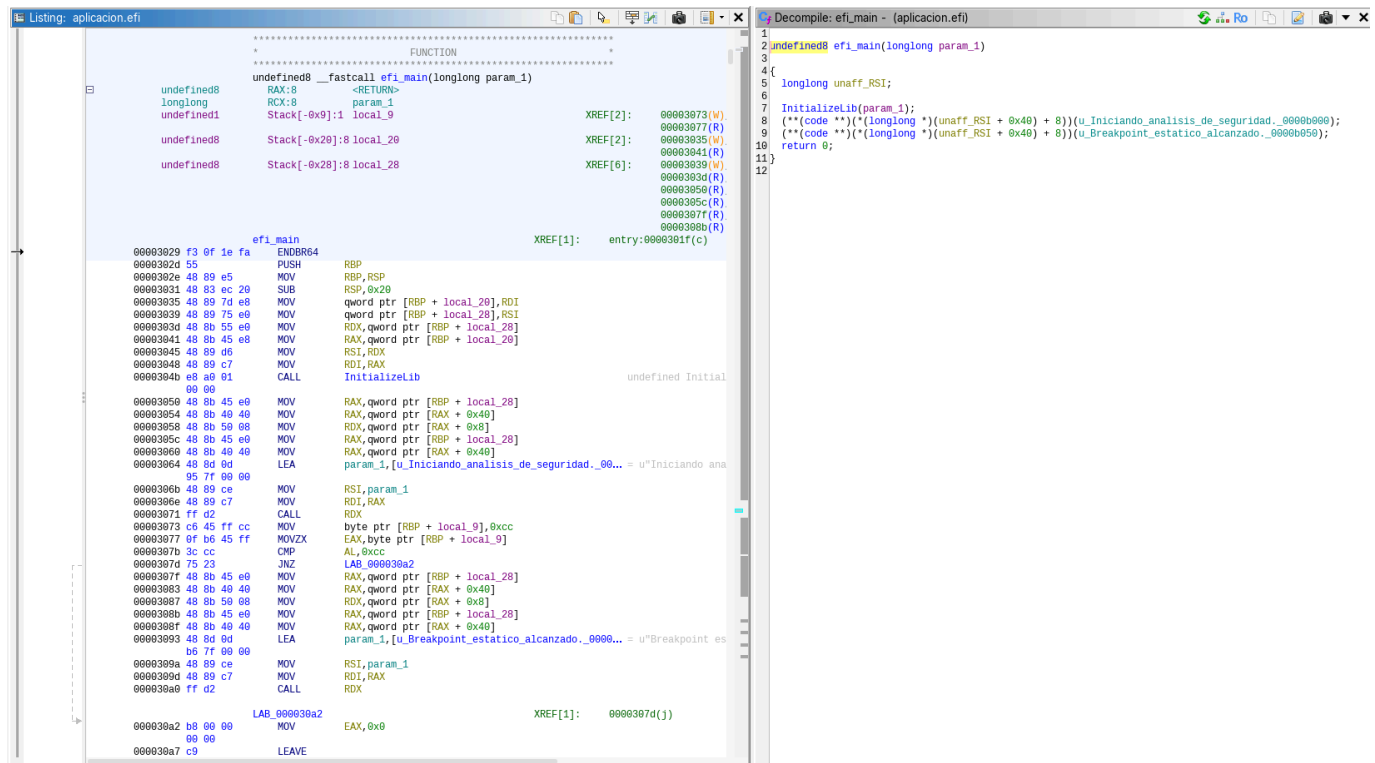
```
file aplicacion.efi
readelf -h aplicacion.efi
```

```
marcos@marcos-HP-Laptop-15-ef2xxx:~/Escritorio/UNC/Sistemas de Computacion/nombreCoolRepo/TP3/TP3.a$ file aplicacion.efi
aplicacion.efi: PE32+ executable (EFI application) x86-64 (stripped to external PDB), for MS Windows, 6 sections
marcos@marcos-HP-Laptop-15-ef2xxx:~/Escritorio/UNC/Sistemas de Computacion/nombreCoolRepo/TP3/TP3.a$ readelf -h aplicacion.so
Encabezado ELF:
Mágico:  7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
Clase:                                ELF64
Datos:                                complemento a 2, little endian
Version:                             1 (current)
OS/ABI:                               UNIX - System V
Versión ABI:                          0
Tipo:                                 DYN (Fichero objeto compartido)
Máquina:                              Advanced Micro Devices X86-64
Versión:                              0x1
Dirección del punto de entrada:       0x3000
Inicio de encabezados de programa:    64 (bytes en el fichero)
Inicio de encabezados de sección:     81664 (bytes en el fichero)
Opciones:                             0x0
Size of this header:                  64 (bytes)
Size of program headers:              56 (bytes)
Number of program headers:            6
Size of section headers:              64 (bytes)
Number of section headers:            21
Section header string table index:    20
marcos@marcos-HP-Laptop-15-ef2xxx:~/Escritorio/UNC/Sistemas de Computacion/nombreCoolRepo/TP3/TP3.a$
```

Capturas de los comandos file y readelf al archivo de aplicación creado

readelf se uso con el archivo .so debido a que el comando no lee archivos .efi

4.5 Análisis en Ghidra



La función **efi_main** muestra claramente cómo una aplicación UEFI interactúa con el firmware sin depender de un sistema operativo. En lugar de usar bibliotecas estándar como printf, accede a la consola mediante la **EFI_SYSTEM_TABLE** y el protocolo ConOut. El análisis en Ghidra permite observar cómo las abstracciones del código C se traducen en accesos indirectos a estructuras y punteros de función. Además, el uso del byte **0xCC** permite vincular el ejemplo con conceptos de bajo nivel y análisis de seguridad, aunque en este caso el byte se compara como dato y no se ejecuta como instrucción.

¿Por qué 0xCC aparece como -52?

Porque 0xCC es un byte hexadecimal que vale 204 si lo interpretas como entero sin signo.

Pero si Ghidra lo interpreta como un char con signo de 8 bits, el rango posible es: -128 a 127.

En complemento a dos:

- 0xCC = 204 unsigned
- 0xCC = -52 signed

Esto importa en ciberseguridad porque el mismo patrón binario puede verse distinto según el tipo de dato que use el descompilador. Si alguien analiza malware, shellcode, opcodes o buffers, puede confundirse si no distingue entre representación signed y unsigned.

5. Trabajo Práctico 3: Ejecución en hardware físico

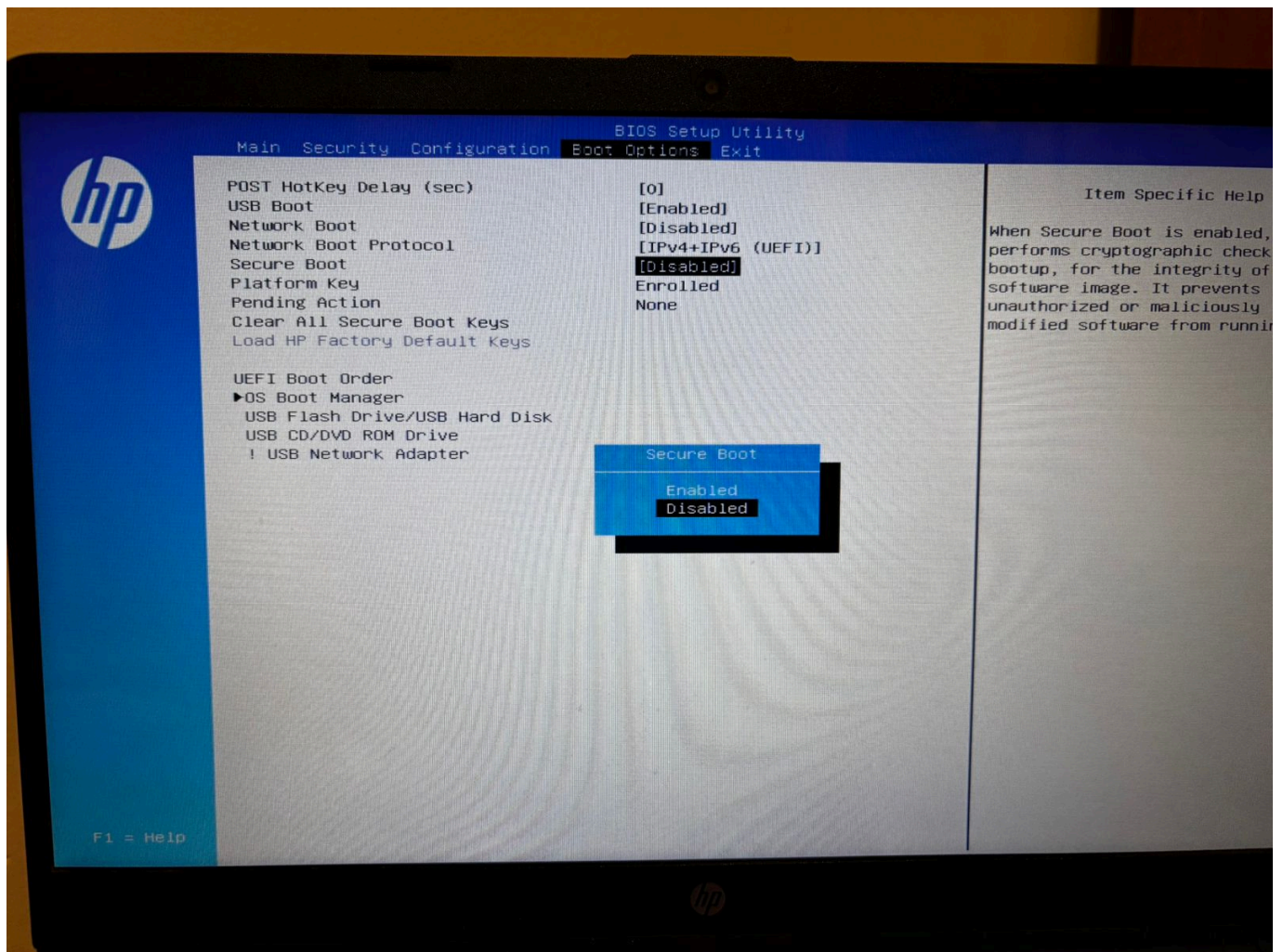
5.1 Preparación del USB

En este paso preparamos el pendrive de arranque utilizando los siguientes comandos:

```
sudo mount /dev/sdb1 /mnt
sudo mkdir -p /mnt/EFI/BOOT
sudo cp aplicacion.efi /mnt/
```

5.2 Configuración del firmware

- Secure Boot: Disabled
- Modo: UEFI Only

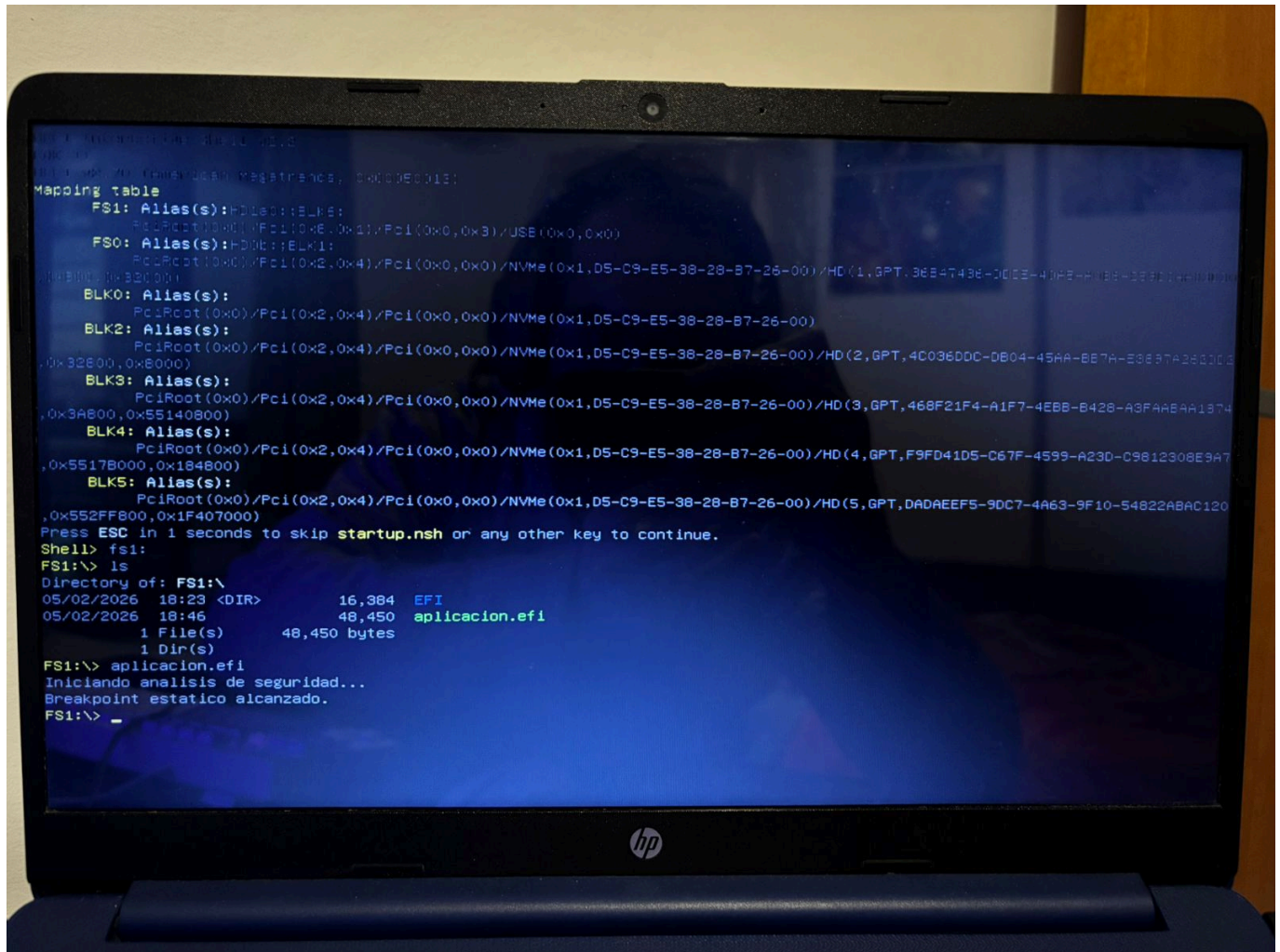


5.3 Ejecución

Por ultimo ejecutamos en Bare Metal la aplicacion

```
FS0:
ls
aplicacion.efi
```


Dando como resultado:



6. Conclusiones

A lo largo de este trabajo pudimos ver que UEFI no es solo un reemplazo moderno del BIOS, sino que en realidad funciona como un entorno completo que se ejecuta antes del sistema operativo. Tiene sus propias reglas, sus servicios y una forma distinta de interactuar con el hardware.

Algo que nos llamó bastante la atención fue el cambio de paradigma respecto al BIOS. En lugar de trabajar directamente con direcciones físicas o hardware específico, UEFI usa un modelo basado en protocolos y handles, lo que hace que el sistema sea mucho más flexible y portable. Al mismo tiempo, esto también implica que las posibles vulnerabilidades ya no están tan ligadas al hardware en sí, sino a cómo se implementan estos mecanismos más abstractos.

Cuando desarrollamos la aplicación UEFI, vimos en la práctica que estamos trabajando en un entorno totalmente distinto al de un sistema operativo tradicional. No hay printf, no hay librerías estándar como las conocemos, y toda interacción pasa por estructuras como la EFI_SYSTEM_TABLE. Esto obliga a entender mejor qué está pasando “por abajo” y cómo se manejan realmente los recursos.

Un aspecto relevante observado es que, a diferencia del desarrollo tradicional, en UEFI los errores pueden tener consecuencias críticas, ya que se ejecuta en etapas muy tempranas del sistema. Esto refuerza la necesidad de entender profundamente el entorno antes de desarrollar o analizar código a este nivel.

Desde el punto de vista de seguridad, quedó bastante claro que UEFI es un punto crítico. Al estar en una etapa tan temprana del arranque, cualquier código que se ejecute ahí tiene un nivel de privilegio muy alto. En particular, las regiones de memoria asociadas a Runtime Services y las variables en NVRAM pueden ser usadas para mantener persistencia, lo que explica por qué son objetivos típicos en ataques como bootkits.

El uso de herramientas como Ghidra también fue interesante porque nos permitió ver cómo el código se traduce a nivel más bajo. Por ejemplo, el caso del 0xCC que aparece como -52 muestra cómo una misma información puede interpretarse de distintas maneras según el contexto, algo que es clave cuando se hace análisis de binarios o ingeniería inversa.

Por último, al ejecutar la aplicación en hardware real, vimos en la práctica el rol de mecanismos como Secure Boot. Entendimos que no es solo una configuración más, sino una parte importante del modelo de seguridad, ya que evita la ejecución de código no firmado en etapas críticas del sistema.
